Campus Food Delivery Management System

Group 7

WIA1002/WIB1002 DATA STRUCTURE

Group Members:

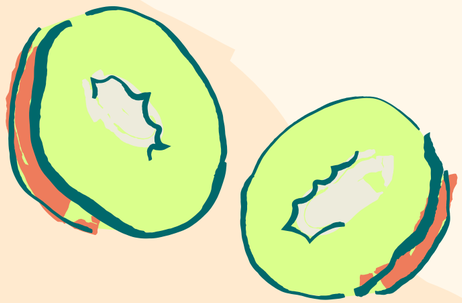
Name	Matric Number
Tan Dao Phang	24076042
An Jun Li	23122616
Yang Pu	24088528
Teow Yan Ping	24063218

Project Overview

The Campus Food Delivery Management System is a hybrid application designed to simulate and manage the logistics of food delivery within a university campus. The system handles order placement, rider dispatching, route optimization, and real-time monitoring.

Key Technical Constraint: This project is built using custom manual data structures without relying on Java's built-in Collections Framework.

- ❌ No java.util.HashMap → ✅ CustomHashMap (Separate Chaining)
- ❌ No java.util.PriorityQueue → ✅ Manual Sorted Array Logic
- ❌ No java.util.ArrayList → ✅ Dynamic Resizing Arrays



TEAM ROLES

An Jun Li: (Graph & Algorithm)	Graph Data Structure, Dijkstra's Algorithm, Path Caching, Map Management
Teow Yan Ping: (Orders & Priority)	Order Queue Implementation, Priority Logic, Order Data Management
Tan Dao Phang: (Dispatch & Integration)	Dispatch Algorithm, Rider Management, Undo/Redo System, System Integration
Yang Pu: (UI & Presentation)	Main Menu, Web Server Integration, Documentation, Final Presentation

System Features & Upgrades

Custom Data Structures

- Implemented CustomHashMap<K,V> from scratch using bucket arrays and linked-list collision handling for O(1) rider/order lookups.

Hybrid Interface

- Console: Robust CLI for standard operations.
- Web Dashboard: A real-time HTML5 Canvas visualization of the graph, showing riders moving between nodes and live order statuses. Built using a custom threaded HttpServer.

Weighted Optimization Dispatch

- Riders are chosen based on a weighted score:
- Score = (1.0 * Distance) + (2.0 * Jobs_Completed) - (0.5 * Idle_Time)

File Persistence

- Ability to Save/Load system state (Riders, Orders, Locations) to .txt files.

Undo/Redo Capability

- Implemented using manual Stack data structures to reverse dispatch actions.

Path Caching

- Stores the result of Dijkstra's algorithm. If a route is requested again, it is retrieved from the cache (O(1)) instead of recalculating (O(V+E)).

Custom Data Structures

CustomHashMap: (CustomHashMap.java)

```java
/**
 * CustomHashMap implements a Hash Table data structure from scratch.
 * Strategy: Separate Chaining with Linked Lists.
 * Time Complexity: Average O(1) for put/get.
 */
public class CustomHashMap<K, V> {

    // Internal node class for the Linked List in each bucket
    private class Entry<K, V> {
        K key;
        V value;
        Entry<K, V> next;

        public Entry(K key, V value) {
            this.key = key;
            this.value = value;
            this.next = null;
        }
    }

    // The array of buckets (Linked Lists)
    private Entry<K, V>[] buckets;
    private int capacity;  // Size of the array
    private int size;      // Total number of key-value pairs

    // Default capacity (Prime number preferred for better distribution)
    private static final int DEFAULT_CAPACITY = 101;

    @SuppressWarnings("unchecked")
    public CustomHashMap() {
        this.capacity = DEFAULT_CAPACITY;
        // Create array of Entry objects
        this.buckets = new Entry[capacity];
        this.size = 0;
    }

    /**
     * Hash function to map key to an index.
     * Uses Java's object hashCode() and modulo operator.
     */
    private int getBucketIndex(K key) {
        int hashCode = key.hashCode();
        // Use Math.abs to ensure positive index
        return Math.abs(hashCode) % capacity;
    }

    /**
     * Put a key-value pair into the map.
     * If key exists, update value. If not, add new entry.
     */
    public void put(K key, V value) {
        int index = getBucketIndex(key);
        Entry<K, V> head = buckets[index];

        // Check if key already exists in the chain
        Entry<K, V> current = head;
        while (current != null) {
            if (current.key.equals(key)) {
                current.value = value; // Update existing value
                return;
            }
            current = current.next;
        }

        // Key not found, insert new node at the HEAD of the chain (faster)
        Entry<K, V> newEntry = new Entry<>(key, value);
        newEntry.next = buckets[index];
        buckets[index] = newEntry;
        size++;
    }

    /**
     * Get value by key.
     * Returns null if key is not found.
     */
    public V get(K key) {
        int index = getBucketIndex(key);
        Entry<K, V> current = buckets[index];

        // Traverse the linked list at this bucket
        while (current != null) {
            if (current.key.equals(key)) {
                return current.value;
            }
            current = current.next;
        }

        return null; // Not found
    }

    /**
     * Check if map contains a key.
     */
    public boolean containsKey(K key) {
        return get(key) != null;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }
}
```

Custom Data Structures

Manual array-based undo/redo stacks: (DispatchSystem.java)

Basic Definitions

```
15          // Manual array-based undo/redo stacks
16          private DispatchAction[] undoStack;
17          private int undoTop;
18          private DispatchAction[] redoStack;
19          private int redoTop;
20          private static final int MAX_STACK_SIZE = 50;
```

Construction Method

```
35              // Initialize undo/redo stacks
36              this.undoStack = new DispatchAction[MAX_STACK_SIZE];
37              this.undoTop = 0;
38              this.redoStack = new DispatchAction[MAX_STACK_SIZE];
39              this.redoTop = 0;
```

Push

```
197             // 8. UPGRADE 3: Push to undo stack (manual array-based)
198             DispatchAction action = new DispatchAction(
199                     nextOrder.getId(), bestRider.getId(),
200                     previousRiderLocation, previousOrderStatus, totalDistance);
201             if (undoTop < MAX_STACK_SIZE) {
202                 undoStack[undoTop++] = action;
203             }
204             redoTop = 0; // Clear redo history on new action
```

Pop(Undo)

```
301             if (undoTop == 0) {
302                 System.out.println("Nothing to undo. Undo stack is empty.");
303                 return;
304             }
305
306             DispatchAction action = undoStack[--undoTop];
338                if (redoTop < MAX_STACK_SIZE) {
339                     redoStack[redoTop++] = action;
```

Pop(Redo)

```
356         public void redoLastDispatch(OrderSystem orderSystem, CampusMap map) {
357             if (redoTop == 0) {
358                 System.out.println("Nothing to redo. Redo stack is empty.");
359                 return;
360             }
361
362             DispatchAction action = redoStack[--redoTop];
```

Stack Empty Check

```
403         public boolean canUndo() {
404             return undoTop > 0;
405         }
406
407         /**
408          * Check if redo is possible
409          */
410         public boolean canRedo() {
411             return redoTop > 0;
412         }
```

Custom Data Structures

Core Call

(DispatchSystem.java)

-Rider ID Lookup

```
 6  public class DispatchSystem {
27      public DispatchSystem() {
28          // Initialize manual arrays
29          this.riderList = new Rider[MAX_RIDERS];
30          this.riderCount = 0;
31
32          // Initialize Custom Map
33          this.riderMap = new CustomHashMap<>();
34
35          // Initialize undo/redo stacks
36          this.undoStack = new DispatchAction[MAX_STACK_SIZE];
37          this.undoTop = 0;
38          this.redoStack = new DispatchAction[MAX_STACK_SIZE];
39          this.redoTop = 0;
40
41          this.totalOrdersDispatched = 0;
42          this.totalDistanceAssigned = 0.0;
43      }
44
45      public void addRider(Rider rider) {
46          if (riderCount < MAX_RIDERS) {
47              riderList[riderCount++] = rider;
48
49              // Add to HashMap for fast lookup
50              riderMap.put(rider.getId(), rider);
51          }
52      }
55       * Get rider by ID - Uses CustomHashMap for O(1) access
56       */
57      public Rider getRider(String id) {
58          // Requirement 3: Fast lookup using Custom Hash Map
59          return riderMap.get(id);
60      }
```

(OrderSystem.java)

-Order ID Lookup

```
20      private CustomHashMap<String, Order> orderMap;
21
22      public OrderSystem() {
23          this.orderQueue = new Order[MAX_ORDERS];
24          this.front = 0;
25          this.rear = 0;
26
27          this.allOrders = new Order[MAX_ORDERS];
28          this.orderCount = 0;
29
30          // Initialize the Custom Map
31          this.orderMap = new CustomHashMap<>();
32      }
33
34      /**
35       * Search for order by ID - Uses CustomHashMap for O(1) access
36       */
37      public Order searchOrder(String orderID) {
38          // Requirement 3: Fast lookup using Custom Hash Map
39          return orderMap.get(orderID);
40      }
41
42      /**
43       * Check if order ID already exists
44       */
45      private boolean orderExists(String orderID) {
46          return searchOrder(orderID) != null;
47      }
50       * Add a new order to the system
51       */
52      public void addOrder(Order order) {
53          if (orderExists(order.getId())) {
54              System.out.println("Error: Order ID " + order.getId() + " already exists.");
55              return;
56          }
57
58          if (orderCount >= MAX_ORDERS) {
59              System.out.println("Error: Maximum order capacity reached.");
60              return;
61          }
62
63          // Add to all orders array (For listing)
64          allOrders[orderCount++] = order;
65
66          // Add to HashMap (For fast lookup)
67          orderMap.put(order.getId(), order);
68
69          // If pending, add to queue
70          if (order.getStatus().equals("Pending")) {
71              enqueue(order);
72          }
73      }
```

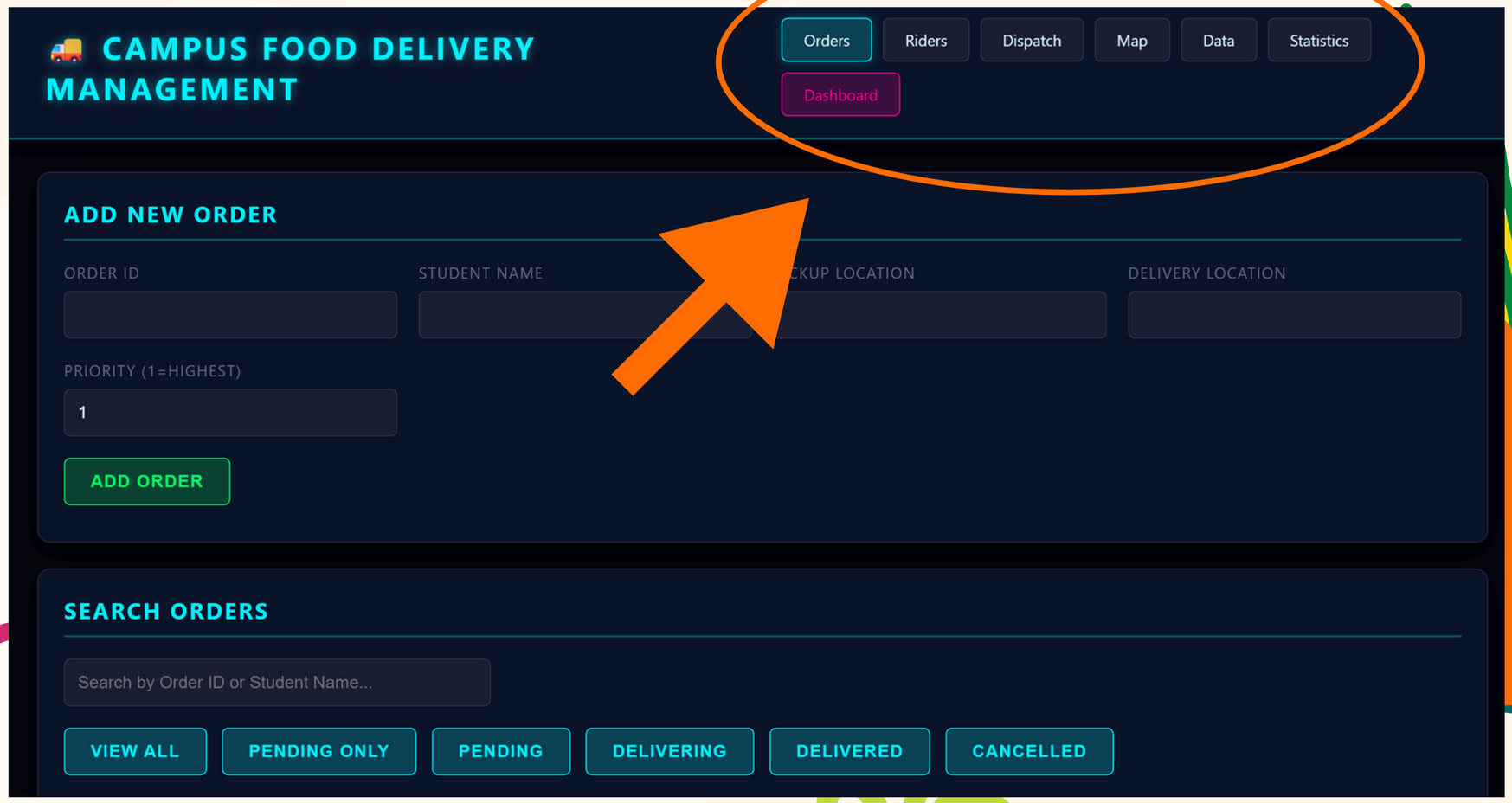
Main Functions

Main Menu

```
        CAMPUS FOOD DELIVERY MANAGEMENT SYSTEM

Initializing Systems... Done!
Loaded 32 locations from data/locations.txt
Loaded 38 routes from data/routes.txt
Loaded 10 riders from data/riders.txt
Loaded 5 orders from data/orders.txt
? System initialized successfully!
? Web Dashboard running at: http://localhost:8080
? Admin Panel: http://localhost:8080/admin.html


            MAIN MENU

  1. Order Management
  2. Rider Management
  3. Dispatch Operations
  4. Campus Map
  5. Save/Load Data
  6. System Statistics
  0. Exit

Enter choice:
```

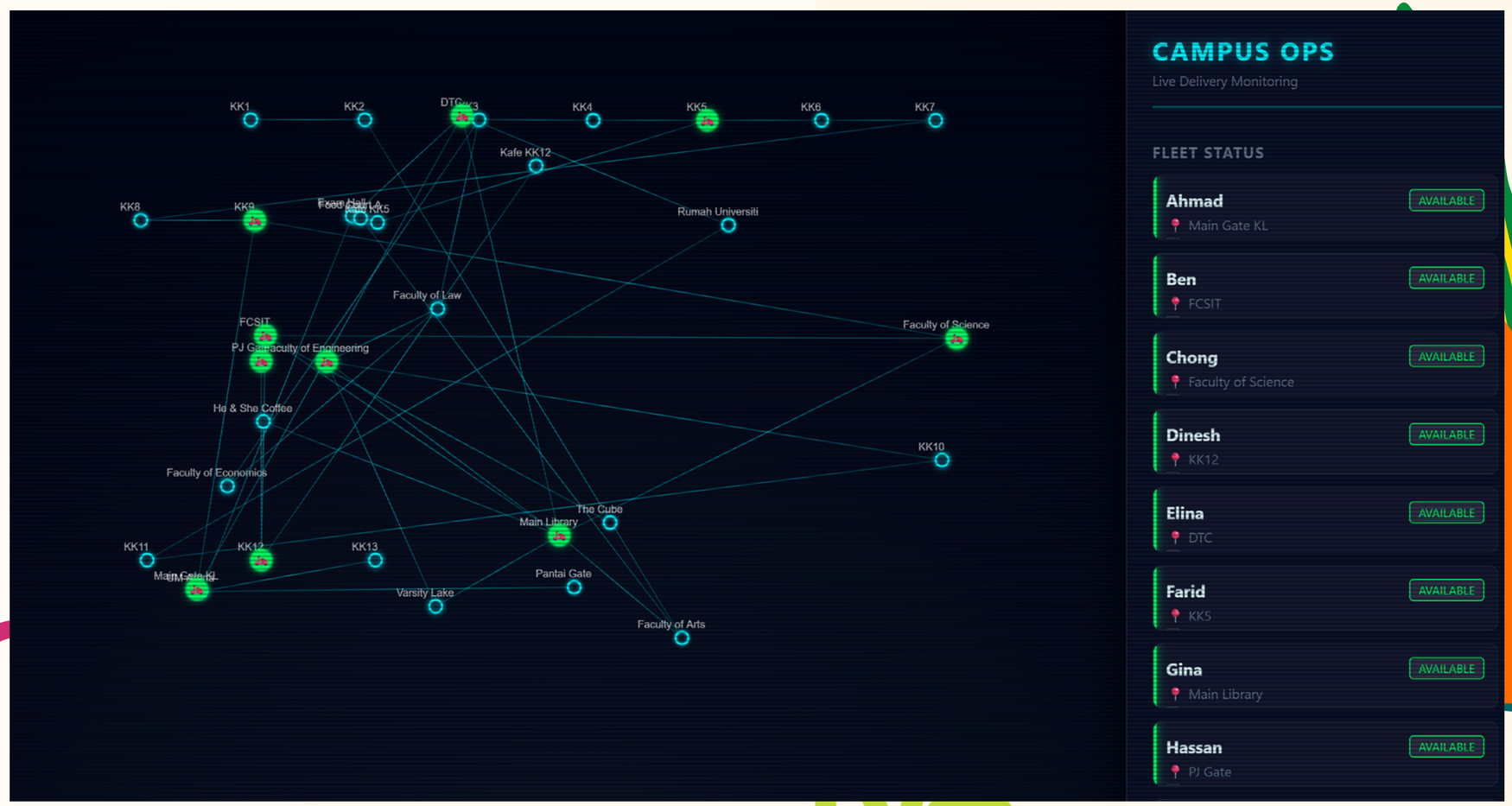
Main Functions

Management System



Main Functions

Campus Map



Requirements Compliance Checklist

Requirement	Implementation Detail	Location
Graph (Adjacency List)	Custom Edge[][] array structure	Graph.java
Priority Queue	Manual sorted insertion logic (Insertion Sort)	OrderSystem.java
Lookup (Dictionary)	Custom Hash Map (Separate Chaining)	CustomHashMap.java
Shortest Path	Dijkstra's Algorithm (Manual Implementation)	Graph.java
Stack/Queue	Manual Array-based Stack (Undo/Redo) and Queue	DispatchSystem.java

Project Walkthrough

Thank you!

--Group 7